Fine-Tuning Foundation Models

A Practical Course for Language, Healthcare & Multimodal AI — and Private Deployment on NVIDIA Hardware

Language fine-tuning (Sanskrit · Tamil · Telugu · Urdu) • Healthcare multimodal (ECG · X-ray · Video) • Reasoning models • NIM / TensorRT-LLM / Triton / Dynamo

INTERVIEW-PREPARATION EDITION

Contents

This book takes you from a pretrained base model to a fine-tuned, evaluated, and privately deployed system — first for language (Sanskrit, Tamil, Telugu, Urdu), then for healthcare signals, images, and video (ECG arrhythmia, X-ray findings, patient-behavior video), then for reasoning ("thinking") models, and finally for serving all of it on your own NVIDIA hardware.

Every chapter mixes three things: the **concept** (what an interviewer wants you to explain clearly), **real code** (what a practitioner actually runs), and short **Interview tips** and **Pitfalls** boxes. Read it front to back for a course, or jump by chapter as a reference.

A note on philosophy that recurs throughout: **the model is rarely the bottleneck — the data and the evaluation are.** If you internalize that, you'll answer most interview questions well before you reach the code.

Part I — Foundations of Fine-Tuning

1. The Landscape: Pretraining, Fine-Tuning, and When to Do Which

1.1 The three stages of a model's life

Modern models are built in stages, and being able to name them crisply is table stakes in an interview.

Pretraining. Start from random weights. Feed enormous amounts of unlabeled text and train with one objective: predict the next token. This is where the model learns grammar, facts, and reasoning patterns. It is expensive (thousands of GPU-hours to millions of dollars) and is almost never something you do from scratch in an applied role.

Supervised fine-tuning (SFT). Start from a pretrained model and train it on curated *(instruction, response)* pairs so it learns a behavior — to follow instructions, answer in a style, or perform a domain task. Cheap by comparison (hours on a single machine with the right techniques).

Preference optimization / alignment. After SFT, nudge the model toward *better* answers using preference data (chosen vs. rejected) via methods like DPO, or toward *verifiably correct* answers via reinforcement learning (RLVR/GRPO — the engine behind reasoning models). This shapes quality, tone, and safety.

A useful mental model:

> Pretraining teaches the *language*. SFT teaches the *task*. Preference tuning teaches *taste*. Reasoning RL teaches *deliberation*.

1.2 Base vs. Instruct models

You will download one of two flavors:

- A **base** (or "completion") model only continues text. Ask it a question and it may continue with *more questions*. It is raw capability with no manners.
- An **instruct** (or "chat") model has already been SFT'd (and usually preference-tuned) to follow instructions and hold a conversation.

Which to fine-tune?

- Fine-tuning an **instruct** model is the gentle, forgiving path: it already follows instructions, and you're nudging its domain or style. Great for most applied tasks and for a fast first result.
- Fine-tuning a **base** model gives you maximum control and avoids fighting a prior chat style, but you must teach it *everything* about the target behavior, which needs more

data. This is the right choice for **continued pretraining** on a new language (Chapter 7).

1.3 When NOT to fine-tune

Interviewers love this question because it separates people who reach for the biggest hammer from people who think about cost and maintenance.

Prefer a cheaper option when it suffices:

- **Prompt engineering** — if a better prompt or few-shot examples solve it, do that first. Zero training cost, instant iteration.
- **Retrieval-Augmented Generation (RAG)** — if the problem is *knowledge* the model lacks (your private documents, your Sanskrit corpus), retrieval usually beats fine-tuning. Fine-tuning teaches *behavior and style*; RAG injects *facts*. This distinction is one of the most important in the whole field.
- **Tool use / function calling** — if the model needs current data or exact computation, give it a tool, don't train it to memorize.

Fine-tune when you need a *durable change in behavior, format, style, or domain skill* that prompting can't reliably produce, or when you need to run a **smaller, cheaper** model at inference by baking in a capability.

> **Interview tip.** A crisp answer to "fine-tune vs. RAG?" is: "*RAG changes what the model knows; fine-tuning changes how it behaves. If the failure is missing facts, retrieve. If the failure is wrong format, style, or task-following, fine-tune. They compose — a fine-tuned model over a RAG pipeline is common.*"

1.4 A taxonomy of fine-tuning methods

- **Full fine-tuning** — update every weight. Best quality ceiling, highest cost (you store gradients + optimizer state for the whole model, roughly 4× the model's memory in fp16/bf16 with Adam). Risk of **catastrophic forgetting**.
- **Parameter-efficient fine-tuning (PEFT)** — freeze most weights, train a small add-on. LoRA and QLoRA dominate. 90–99% of the quality for a few percent of the cost; the base is frozen, so forgetting is milder. This is what you'll use on a single machine.
- **Continued / domain-adaptive pretraining** — keep the next-token objective but on domain or new-language text, to move the model's *knowledge*, not just its behavior. Often precedes SFT for a new language.

The rest of Part II makes each of these concrete.

---

2. Transformers & Tokenization for Multilingual Work

You don't need to re-derive backprop in an interview, but you must speak fluently about attention and — especially for Indic and Perso-Arabic languages — **tokenization**, which is where multilingual projects quietly succeed or fail.

2.1 The transformer, in one page

A decoder-only language model is a stack of identical **blocks**. Each block does two things:

1. **Causal self-attention** — every token position looks back at earlier positions and pulls in information from whichever it finds relevant. Concretely, each position emits a *query*, *key*, and *value*; the dot product of query·key (scaled, masked so you can't see the future, then softmaxed) gives weights used to average the values.
2. **A feed-forward MLP** — each position independently "thinks" about what it gathered.

Wrapped around each with **residual connections** (add the input back, giving gradients a clean highway) and **layer normalization** (keep activations numerically stable). Below the stack sit two embeddings — one for *token identity*, one for *position* — and above it a linear head projecting to a probability over the vocabulary. Training minimizes cross-entropy on next-token prediction. That single objective, at scale, produces everything else.

```
input tokens → [token emb + position emb]
             → N × { LayerNorm → Self-Attention → +residual
                     LayerNorm → MLP            → +residual }
             → LayerNorm → Linear head → next-token probabilities
```

2.2 Tokenization: the multilingual battleground

A **tokenizer** maps text to integer IDs the model consumes. Almost all modern LLMs use **subword** tokenization (Byte-Pair Encoding or Unigram) trained to give common sequences short encodings. The critical metric for multilingual work is **fertility**: the average number of tokens per word.

Here's the problem. Most popular tokenizers were trained on English-heavy corpora. Feed them Devanagari (Sanskrit/Hindi), Tamil, Telugu, or Urdu (Perso-Arabic), and they fragment each word into many tokens — sometimes falling back to raw bytes. High fertility has three bad consequences:

- **Shorter effective context** — a 4,096-token window holds far less Tamil than English.
- **Slower, costlier** training and inference — more tokens per sentence.
- **Wasted model capacity** — the model spends effort reassembling orthography instead of learning meaning.

Two more script-specific traps:

- **Devanagari (Sanskrit, Hindi, Marathi)** is an *abugida*: a visible syllable (an *akshara*) is often several Unicode code points — a consonant plus vowel signs, or consonant clusters joined by a *virama* (्). Naive character splitting shatters syllables; you want grapheme-cluster awareness. (This is the exact lesson many practitioners first meet building a small GPT.)
- **Urdu** uses the **Perso-Arabic** script: right-to-left, cursive, with contextual letter forms and usually-omitted short-vowel diacritics. Normalization (Unicode NFC, consistent handling of Arabic vs. Urdu code points like ي/ی and ك/ک) matters enormously, and RTL display can hide data-cleaning bugs.

Let's measure fertility directly — the kind of quick diagnostic you'd run on day one and could sketch on a whiteboard:

```
from transformers import AutoTokenizer

samples = {
    "English": "Knowledge gives humility.",
    "Hindi":   "विद्या विनय देती है।",
    "Sanskrit":"विद्या ददाति विनयम्।",
    "Tamil":   "அறிவு பணிவைத் தரும்.",
    "Telugu":  "జ్ఞానం వినయాన్ని ఇస్తుంది.",
    "Urdu":    "علم عاجزی دیتا ہے۔",
}

def fertility(tokenizer_id):
    tok = AutoTokenizer.from_pretrained(tokenizer_id)
    print(f"\n== {tokenizer_id} ==")
    for lang, text in samples.items():
        n_words = len(text.split())
        n_tokens = len(tok(text, add_special_tokens=False)["input_ids"])
        print(f"  {lang:9} words={n_words:2}  tokens={n_tokens:2}  "
              f"fertility={n_tokens / max(n_words,1):.2f}")

# Compare an English-centric tokenizer against an Indic-aware one:
fertility("meta-llama/Llama-3.2-1B")     # typically high fertility on Indic scripts
fertility("sarvamai/sarvam-1")           # Indic-optimized: fertility ~1.4–2.1
```

You will typically see an English-centric tokenizer produce 3–6× more tokens per Indic word than an Indic-tuned tokenizer such as Sarvam's (reported fertility ≈ 1.4–2.1 across ten Indian languages). That single measurement often decides your base-model choice.

> **Interview tip.** If asked "what's hard about fine-tuning for Tamil/Urdu specifically?", lead with tokenization and fertility, mention script-specific normalization (abugida syllables; Perso-Arabic RTL and diacritics), then note the fix: pick an Indic/multilingual base, or extend the tokenizer + do continued pretraining (Chapter 7).

2.3 Choosing a base model for a low-resource language

Decision factors, roughly in order:

1. **Tokenizer coverage** of your script (measure fertility — don't guess).
2. **Pretraining data** in or near your language (a model that saw Hindi transfers to Sanskrit better than one that saw neither).
3. **Size vs. your hardware** — bigger is better for quality but slower to iterate; start small.
4. **License** — can you use and redistribute it? (Gemma and Llama have specific terms; Qwen and many others are Apache-2.0.)

Concrete mid-2026 options worth knowing by name: **Sarvam-1** (2B, Indic-specialized, efficient Indic tokenizer, a *base* model good for continued pretraining), **Gemma 3** (strong multilingual; the substrate Sarvam used for a 22-language Indic translator that *includes Sanskrit*), **Qwen** family (excellent tooling support, permissive license, decent multilingual), and **Llama 3.x** (broad ecosystem). For a pure learning exercise, a small Qwen or Gemma gets you iterating in minutes.

Part II — Fine-Tuning Language Models

3. Data: The Real Work

Models are commodities; **datasets are the moat.** Interviewers probe data because it's where applied skill lives.

3.1 What SFT data looks like

The unit is a pair. Two common shapes:

Prompt–completion:

```
{"prompt": "Translate to English: सत्यं वद।", "completion": "Speak the truth."}
```

Conversational (messages):

```
{"messages": [
  {"role": "system", "content": "You are a Sanskrit tutor."},
  {"role": "user", "content": "What does 'धर्म' mean?"},
  {"role": "assistant", "content": "'Dharma' means righteous duty or moral law..."}
]}
```

Keeping prompt and completion *separate* matters: it lets the trainer compute loss on the **completion only** (score the answer, not the parroted question). More on that in Chapter 5.

3.2 Where data comes from (and the quality bar)

For a low-resource language you rarely have ready-made instruction data. Your options, best-effort first:

- **Human-curated pairs** — highest quality, slowest. Even a few hundred excellent, varied examples move a model meaningfully.
- **Templated construction** from structured sources — e.g., a Sanskrit verse + known translation → a "translate this" pair; a dictionary → "define this word" pairs.
- **Synthetic generation ("distillation from a teacher")** — prompt a strong model to draft translations/explanations/Q&A, then **have a human review**. This is standard practice: one published Sanskrit effort generated ~11,000 synthetic Q&A pairs this way; Sarvam's post-training similarly leaned on large synthetic SFT sets.

Quality rules that matter more than volume:

- **Diversity of task types** beats raw count. A dataset that is 100% "translate" produces a translator, not an assistant. Mix translate / explain / complete / define / converse.
- **Deduplicate** aggressively; near-duplicates inflate your metrics and encourage memorization.
- **Decontaminate** — make sure eval examples aren't in the training set. Interviewers *will* ask how you prevent train/test leakage.

- **Balance** — watch for length bias (models learn "longer = better" if your good answers are all long) and topic skew.

```
# A tiny, reusable quality pass: exact + near-duplicate removal and a length audit.
import json, hashlib
from collections import Counter

def load(path):
    return [json.loads(l) for l in open(path, encoding="utf-8")]

def dedup(rows, key=lambda r: r["prompt"] + "\u241f" + r["completion"]):
    seen, out = set(), []
    for r in rows:
        h = hashlib.sha256(key(r).encode("utf-8")).hexdigest()
        if h not in seen:
            seen.add(h); out.append(r)
    return out

rows = dedup(load("sft.jsonl"))
lens = Counter(len(r["completion"]) // 50 * 50 for r in rows)  # length histogram (buckets
of 50 chars)
print(f"{len(rows)} unique examples; completion-length buckets:
{dict(sorted(lens.items()))}")
```

> **Pitfall.** Synthetic data inherits the *teacher's* mistakes and biases — and in a language the teacher is weak at (Sanskrit!), those mistakes can be subtle and confidently wrong. Always sample and review; never train on unreviewed synthetic data for a domain where you can't yet evaluate.

3.3 Chat templates

Every instruct model was trained with an exact special-token layout marking system/user/assistant turns — its **chat template**, stored in the tokenizer. At training and inference you must format prompts with that same template, or the model gets confused even though its weights are fine.

```
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-1.5B-Instruct")

messages = [{"role": "user", "content": "Translate: सत्यं वद।"}]
prompt = tok.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
print(prompt)   # shows the model's exact turn markers
```

> **Interview tip.** "My fine-tuned model got dumber" is, nine times out of ten, a chat-template or loss-masking mismatch, not a training failure. Say that and you'll sound experienced.

---

4. Parameter-Efficient Fine-Tuning: LoRA & QLoRA

This is the single most important practical technique in modern applied fine-tuning, and a near-guaranteed interview topic. Be able to explain *why* it works, not just call the API.

4.1 The memory problem it solves

Full fine-tuning of an N-parameter model with the Adam optimizer in mixed precision needs roughly: the weights (2 bytes each in bf16), the gradients (2 bytes), and Adam's two moment estimates (often 4 + 4 bytes in fp32). That's on the order of **16 bytes per parameter** — about 16 GB for a 1B model, ~112 GB for a 7B model — before activations. That's why full fine-tuning needs clusters.

4.2 The LoRA idea

Low-Rank Adaptation (LoRA) freezes every original weight matrix W and, beside it, inserts two small matrices A (shape $d \times r$) and B (shape $r \times d$) with a small **rank** r (e.g., 8–32). Only A and B train. Their product is a low-rank "correction" added to the frozen weight:

```
h = W x + (alpha / r) · B (A x)
     └─frozen─┘   └─────trainable─────┘
```

Why does something so small suffice? Because *adapting behavior* doesn't require rewriting what the model knows — it only needs to steer activations along a few directions. Empirically, the update needed to specialize a model is **low-rank**, so a rank-16 adapter captures most of it. You end up training well under 1% of parameters, and because W is untouched, catastrophic forgetting is mild and you can keep many small adapters for one base.

The knobs:

- `r` (rank) — adapter capacity. Higher = more expressive, more memory. 8–16 for style/task; 32–64 for a big domain shift.
- `lora_alpha` — a scaling factor; effective strength ≈ `alpha / r`. A common heuristic is `alpha = 2·r`.
- `lora_dropout` — regularization, often 0.05.
- `target_modules` — which matrices get adapters. Attention projections (`q,k,v,o`) plus the MLP (`gate,up,down`) is the strong default; `"all-linear"` targets them all robustly across model families.

4.3 QLoRA: LoRA on a quantized base

The frozen base is never updated — so why store it at full precision? **QLoRA** loads the base in **4-bit** (the `nf4` format, tuned for weight distributions; optional double-quantization squeezes a bit more), while the LoRA adapters stay in bf16. Memory for the base drops ~4×, so a 7B model fine-tunes on a single 16 GB GPU, and on a 128 GB unified-memory machine you can reach ~70B. The compute during the forward/backward is done by de-quantizing on the fly to a bf16 compute dtype.

```
import torch
from transformers import BitsAndBytesConfig

qlora_4bit = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",          # 4-bit NormalFloat, tuned for weights
    bnb_4bit_use_double_quant=True,     # quantize the quantization constants too
    bnb_4bit_compute_dtype=torch.bfloat16,  # bf16 compute (esp. on Blackwell; avoid fp16)
)
```

4.4 Defining a LoRA config in code

```
from peft import LoraConfig

lora = LoraConfig(
    r=16,
    lora_alpha=32,                  # effective scale alpha/r = 2.0
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules="all-linear",    # robust across Qwen/Gemma/Llama/Sarvam
)
```

> **Interview tip.** If asked "LoRA vs. QLoRA vs. full fine-tuning — how do you choose?": *full* when you have the hardware and need the last few points of quality or a large behavior change; *LoRA* as the default for most tasks; *QLoRA* when memory-bound (biggest base that fits, small speed cost from de-quantization). And mention that LoRA adapters are tiny (tens of MB), swappable, and mergeable back into the base for zero-overhead serving.
>
> **Pitfall.** If `print_trainable_parameters()` reports ~100% trainable, your adapter didn't attach — you forgot to pass the PEFT config, or targeted the wrong modules. Always print and eyeball that it's a fraction of a percent.

5. Supervised Fine-Tuning with TRL — A Complete Pipeline

Now we assemble Chapters 3–4 into a runnable SFT pipeline using Hugging Face **TRL** (the library whose `SFTTrainer` industrializes the training loop), **PEFT** (LoRA), and **transformers**.

5.1 The SFT objective

SFT uses the **exact same** token-level cross-entropy as pretraining — "how surprised was the model by the next token?" — with one crucial addition: with prompt–completion data and **completion-only loss**, the trainer *masks the prompt tokens* so the model is scored only on producing the answer. You don't want to reward it for re-typing the question; you want it good at the response. Everything else — predict, measure surprise, backpropagate, step — is the ordinary training loop.

5.2 The full script

```python
"""sft_train.py — LoRA/QLoRA supervised fine-tuning for a language task."""
import argparse, torch
from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig
from trl import SFTTrainer, SFTConfig

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--model", default="Qwen/Qwen2.5-1.5B-Instruct")
    ap.add_argument("--data",  default="sft.jsonl")      # {"prompt":..., "completion":...}
    ap.add_argument("--out",   default="./adapter")
    ap.add_argument("--qlora", action="store_true")
    args = ap.parse_args()
    use_cuda = torch.cuda.is_available()

    # 1) Data: prompt/completion columns trigger completion-only loss.
    ds = load_dataset("json", data_files=args.data, split="train")

    # 2) Optional 4-bit base for QLoRA.
    quant = None
    if args.qlora:
        quant = BitsAndBytesConfig(
            load_in_4bit=True, bnb_4bit_quant_type="nf4",
            bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=torch.bfloat16)

    # 3) Tokenizer + base model.
    tok = AutoTokenizer.from_pretrained(args.model)
    if tok.pad_token is None:
        tok.pad_token = tok.eos_token
    model = AutoModelForCausalLM.from_pretrained(
        args.model, quantization_config=quant,
        dtype=torch.bfloat16 if use_cuda else torch.float32,
        device_map="auto" if use_cuda else None)

    # 4) LoRA adapters.
    lora = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                      bias="none", task_type="CAUSAL_LM",
                      target_modules="all-linear")

    # 5) Training config. bf16 on GPU (never fp16 on Blackwell). Effective batch = 2*8.
    cfg = SFTConfig(
        output_dir=args.out, num_train_epochs=3,
        per_device_train_batch_size=2, gradient_accumulation_steps=8,
        learning_rate=2e-4, lr_scheduler_type="cosine", warmup_ratio=0.03,
        logging_steps=5, max_length=1024,
        completion_only_loss=True, gradient_checkpointing=True,
        bf16=use_cuda, report_to="none")

    # 6) Train. SFTTrainer applies LoRA, masks prompts, runs the loop.
    trainer = SFTTrainer(model=model, args=cfg, train_dataset=ds,
                         peft_config=lora, processing_class=tok)
    trainer.model.print_trainable_parameters()   # sanity: should be <1%
    trainer.train()
    trainer.save_model(args.out); tok.save_pretrained(args.out)

if __name__ == "__main__":
    main()
```

Run it: `python sft_train.py --qlora`. The adapter lands in `./adapter` at tens of MB.

5.3 Hyperparameters that actually matter

- **Learning rate** — LoRA tolerates higher LRs than full fine-tuning; `1e-4` – `3e-4` is a good band. Too high → loss spikes/instability; too low → nothing learns.
- **Epochs** — 1–3 for a decent dataset. More epochs on a *small* set overfits fast (you'll see val loss turn up).
- **Effective batch size** = `per_device_batch × grad_accum × num_gpus`. Use gradient accumulation to simulate large batches on small memory.
- **Max sequence length** — bumps memory quadratically-ish via attention; set it to your data's real needs, not blindly high.
- **Gradient checkpointing** — trades ~20–30% compute for a large memory saving; almost always on for single-GPU work.

5.4 Watching training

Track **train loss** (should fall then flatten) and, on a held-out split, **val loss** (should track train, then diverge upward when you start overfitting). A quick sanity number: an untrained model's loss ≈ `ln(vocab_size)` (pure guessing); watching it drop below that confirms learning. If loss is flat from step 1, your LR is too low, your labels are wrong, or your adapter didn't attach.

---

6. Preference Optimization: DPO and ORPO

SFT teaches the model to produce *a* good answer. **Preference optimization** teaches it to prefer the *better* of two answers — sharpening quality, helpfulness, tone, and safety. It's the step that turns a competent model into a pleasant one.

6.1 From RLHF to DPO

Classic **RLHF** trains a separate reward model on human preferences, then optimizes the policy against it with PPO — powerful but fiddly (a reward model, a reference model, an RL loop). **Direct Preference Optimization (DPO)** achieves the same goal with a simple classification-style loss and **no separate reward model**: it directly raises the model's likelihood of *chosen* responses over *rejected* ones, regularized by a frozen reference model so it doesn't drift too far. For a single practitioner, DPO (or its reference-free cousin **ORPO**, which folds preference into the SFT stage) is the practical choice.

Preference data shape:

```
{"prompt": "Explain 'karma' briefly.",
 "chosen": "Karma is the principle that intentional actions carry moral consequences...",
 "rejected": "karma is a hindu thing about luck idk"}
```

6.2 DPO in code

```
import torch
from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig
from trl import DPOTrainer, DPOConfig

ds  = load_dataset("json", data_files="prefs.jsonl", split="train")  # prompt/chosen/
rejected
tok = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-1.5B-Instruct")
model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-1.5B-Instruct", dtype=torch.bfloat16, device_map="auto")

cfg = DPOConfig(output_dir="./dpo-adapter", beta=0.1,   # beta = how hard to push vs.
reference
                per_device_train_batch_size=2, gradient_accumulation_steps=8,
                learning_rate=5e-6, num_train_epochs=1, bf16=True, report_to="none")

trainer = DPOTrainer(model=model, args=cfg, train_dataset=ds, processing_class=tok,
                     peft_config=LoraConfig(r=16, lora_alpha=32, task_type="CAUSAL_LM",
                                            target_modules="all-linear"))
trainer.train()
```

`beta` is the key knob: small `beta` lets the model move further from the reference (stronger preference shaping, more risk of degeneration); large `beta` keeps it close. Learning rates for DPO are much smaller than SFT (`~5e-6`).

> **Interview tip.** Be ready to sequence the pipeline: **SFT first, then DPO.** DPO assumes a model that already produces reasonable answers; running it on a base model with no SFT usually disappoints. And note DPO's big selling point: no reward model, no RL loop, so it's stable and cheap.

---

7. Continued Pretraining & Vocabulary Extension for a New Language

Sometimes SFT isn't enough. If the base model barely saw your language — say you want deep **Sanskrit** or **Telugu** fluency and the base is English-centric — SFT will polish a surface it doesn't really have. The fix is **continued (domain-adaptive) pretraining**: keep the next-token objective, but train on a large corpus of the target language, *then* SFT.

7.1 The recipe

1. **Assemble a clean monolingual corpus** in the target language (your OCR'd texts, public corpora, filtered web text). This is the bottleneck — see the healthcare/data themes throughout.
2. *(Optional but powerful)* **Extend the tokenizer** so the new script isn't fragmented.
3. **Continue pretraining** (often with LoRA to save memory, though full is better if you can afford it) on the corpus.
4. **SFT** on instruction pairs, then optionally **DPO**.

7.2 Vocabulary extension

If fertility is terrible, train new subword tokens on your corpus and add them to the tokenizer; then **resize the model's embedding matrix** so the new tokens have (initially random, then trained) vectors.

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tok   = AutoTokenizer.from_pretrained("meta-llama/Llama-3.2-1B")
model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-3.2-1B")

# Suppose you trained a SentencePiece/BPE model on Telugu and extracted new pieces:
new_tokens = ["▁జ్ఞానం", "▁వినయం", "▁ధర్మం"]      # illustrative
added = tok.add_tokens(new_tokens)
model.resize_token_embeddings(len(tok))            # grow the embedding + output head
print(f"added {added} tokens; new vocab size = {len(tok)}")

# Good practice: initialize each new token's embedding as the mean of its
# sub-token pieces' embeddings, so it starts from a sensible point rather than noise.
```

After extension, the new embeddings are untrained, so you **must** do continued pretraining to make them meaningful. This is exactly how projects build strong models for a specific Indic language on top of a general base.

Pitfall. Extending vocabulary and then only doing a light SFT gives worse results than not extending at all — the fresh embeddings never learn. Budget real continued-pretraining compute, or skip extension and just pick an already-Indic-tuned base (Sarvam-1) instead.

Interview tip. The clean framing: "*SFT changes behavior; continued pretraining changes competence. For a language the base barely knows, you need the latter first — possibly with tokenizer extension — or you're just teaching manners to a model that can't speak the language.*"

---

8. Evaluating Language Models

If you take one thing from this book into an interview: **you cannot improve what you cannot measure**, and evaluation is where most projects are weakest. Expect deep questions here.

8.1 Intrinsic vs. task metrics

- **Perplexity** (exp of the mean loss) — cheap, good for tracking language modeling on held-out text, but *not* a measure of usefulness. Lower is better; comparable only within the same tokenizer.
- **Task metrics** — pick the metric that matches the task: BLEU/chrF/COMET for translation (chrF and COMET are far better than BLEU for morphologically rich Indic languages), ROUGE for summarization, exact-match/F1 for extractive QA, accuracy for classification.

8.2 LLM-as-judge

For open-ended generation (explanations, chat), automatic n-gram metrics are weak. The modern practice is **LLM-as-judge**: prompt a strong model with a rubric to score or compare outputs. Cheap, fast, and correlates reasonably with humans — but bias-prone (position bias, length bias, self-preference). Mitigate by randomizing order, controlling for length, and validating the judge against a small human-labeled set.

```python
# Sketch: pairwise LLM-as-judge with position-bias control.
def judge(prompt, ans_a, ans_b, call_model):
    rubric = ("You are grading Sanskrit-to-English translations. "
              "Score accuracy, fluency, completeness. Reply only 'A', 'B', or 'tie'.")
    # Ask twice with swapped order; only count if consistent, else 'tie'.
    r1 = call_model(f"{rubric}\nSource:{prompt}\nA:{ans_a}\nB:{ans_b}\nBetter?")
    r2 = call_model(f"{rubric}\nSource:{prompt}\nA:{ans_b}\nB:{ans_a}\nBetter?")
    if r1 == "A" and r2 == "B": return "a"
    if r1 == "B" and r2 == "A": return "b"
    return "tie"
```

8.3 Benchmarks for Indic languages

Name-drop these to show domain awareness: **IndicGLUE**, **IndicXTREME**, and the **AI4Bharat** ecosystem for Indic NLU/NLG; **Flores-200** for translation across 200 languages (includes Indic + Urdu). For Sanskrit specifically, evaluation is genuinely hard and under-served — which, as noted, is an opportunity, not just a warning.

8.4 The evaluation discipline

- **Hold out** a test set *before* training and never train on it (decontamination).

- **Report confidence** — small eval sets are noisy; report variation across seeds or bootstrap confidence intervals rather than a single lucky number.
- **Evaluate what you deployed** — the model *plus* its prompt template *plus* any RAG. A weight-only benchmark can look great while the deployed system fails on template mismatch.

Part III — Healthcare & Regulated-Domain Fine-Tuning

9. Working with Healthcare Data

Everything from Part II still applies — but healthcare adds privacy law, safety stakes, and nasty data statistics. This is where interviewers for a *healthcare* role will spend real time.

9.1 Privacy, PHI, and the law

Protected Health Information (PHI) is any health data that can identify a person. In the US, **HIPAA** governs it (the Privacy and Security Rules); the EU adds **GDPR**. Practical consequences for an ML engineer:

- **De-identification** before training. HIPAA's *Safe Harbor* method lists 18 identifiers to remove (names, dates finer than year, MRNs, geographies smaller than state, device IDs, etc.); the *Expert Determination* method certifies statistically low re-identification risk. For clinical *text*, you run a PHI-scrubbing NER step first.
- **Data residency & isolation** — one of the strongest reasons this whole book ends on *private, on-prem* deployment: PHI often legally cannot leave the institution or go to a public API. Fine-tuning and serving on your own NVIDIA hardware keeps data in the building.
- **Auditability & consent** — track data lineage and the legal basis for use; regulated settings demand it.
- **Business Associate Agreements (BAAs)** with any vendor that touches PHI.

```python
# PHI scrubbing sketch. In practice use a clinical de-id tool (e.g., Philter,
# spaCy medspaCy, or a fine-tuned NER) — regex alone is NOT sufficient.
import re
PATTERNS = {
    "MRN":   r"\b\d{6,10}\b",
    "PHONE": r"\b\d{3}[-.\s]?\d{3}[-.\s]?\d{4}\b",
    "DATE":  r"\b\d{1,2}/\d{1,2}/\d{2,4}\b",
    "SSN":   r"\b\d{3}-\d{2}-\d{4}\b",
}
def scrub(text):
    for tag, pat in PATTERNS.items():
        text = re.sub(pat, f"[{tag}]", text)
    return text  # then ALSO run a trained NER for names/addresses regex can't catch
```

> **Interview tip.** When asked "how would you handle patient data for fine-tuning?", lead with de-identification (Safe Harbor vs. Expert Determination), then data isolation / on-prem, then auditability and BAAs. Mentioning that privacy is a *primary driver* of the on-prem NVIDIA deployment ties the whole system together.

9.2 The data statistics that hurt

- **Class imbalance** — the pathological class is usually rare (most X-rays are normal; most heartbeats aren't the dangerous arrhythmia). Accuracy is a *trap*: a model that always says "normal" can score 98%. Use **precision/recall, F1, AUROC, and**

especially AUPRC (better than AUROC under heavy imbalance), and techniques like class weighting, focal loss, and careful resampling.

- **Label noise & inter-rater disagreement** — even expert labels disagree. Know your labels' provenance and agreement rate.
- **Distribution shift** — a model trained on one hospital's scanner/population often degrades on another. Evaluate on **external** data and monitor drift in production.
- **Small data** — clinical datasets are small and expensive, which is exactly why **transfer learning / PEFT / foundation models** dominate healthcare: they need far less task-specific data.

---

10. Clinical LLMs & Medical Foundation Models

10.1 The landscape

Rather than training clinical models from scratch, you start from a medical foundation model and fine-tune. Key ones to know by name:

- **MedGemma** (built on **Gemma 3**, 4B and 27B) — open medical **vision-language** models that read images *and* text. Fine-tuning MedGemma reaches near-specialist performance on tasks like chest-X-ray report generation and pneumothorax classification, while retaining general ability. Its image understanding comes from **MedSigLIP**, a ~400M medical vision encoder (a medically-tuned SigLIP) usable on its own for classification/retrieval.
- **Clinical text models** and de-identified corpora (e.g., **MIMIC**-derived datasets) for discharge summaries, coding, and medical QA.
- **Domain embedding models** (e.g., MedImageInsight for imaging) for retrieval and lightweight classification.

10.2 Fine-tuning a clinical LLM

Mechanically, this is Chapter 5 with healthcare data: SFT (often QLoRA) on *(clinical instruction, expert response)* pairs — summarize this note, extract the diagnoses, answer this patient question safely. What changes is the **guardrails and evaluation** (next chapter), not the training loop.

> **Interview tip.** If asked "would you fine-tune a general LLM or start from MedGemma for a radiology task?": start from the medical foundation model — it already encodes medical image/text priors, so you need far less data and get better calibration, *and* you can still LoRA-fine-tune it for your specific institution. Falling back to a general model is justified only if licensing or modality coverage forces it.

---

11. Safety, Calibration & Evaluation in Clinical AI

A clinically deployed model is a **medical device-adjacent** system; "it usually works" is not a bar you can ship at.

11.1 Calibration and uncertainty

A model that says "90% pneumonia" should be right ~90% of the time at that confidence. Measure with **reliability diagrams** and **Expected Calibration Error (ECE)**; fix with **temperature scaling** (a one-parameter post-hoc method). Provide **abstention** — let the model say "uncertain, refer to clinician" below a confidence threshold, tuned to the clinically acceptable miss rate.

11.2 The metrics clinicians care about

Sensitivity (recall) and specificity, at an **operating point** chosen for the clinical cost of false negatives vs. false positives — missing a cancer is not symmetric with a false alarm. Report **AUROC and AUPRC** across the curve, plus performance **stratified by subgroup** (age, sex, site, device) to surface bias. A single accuracy number is a red flag in a clinical interview.

11.3 Hallucination and grounding

For clinical *text*, unsupported statements can be dangerous. Prefer **RAG grounding** in the patient's record and cite sources; constrain generation; and evaluate **faithfulness** (does every claim trace to evidence?) not just fluency. For report generation, domain metrics like **RadGraph F1** measure whether the *clinical entities and relations* are right, not just the wording.

11.4 Regulation and process

Know the vocabulary: **FDA SaMD** (Software as a Medical Device) and the concept of a **Predetermined Change Control Plan** for models that update; the EU **MDR** and **AI Act** risk tiers; and the need for **human-in-the-loop** for high-risk decisions. You won't run a regulatory submission in an ML interview, but showing you know these exist — and that they shape how/when you're allowed to update a deployed model — signals maturity.

> **Interview tip.** A strong closing line for any clinical-AI answer: "*I'd treat the model as one component of a monitored, human-in-the-loop clinical workflow — with calibration, subgroup evaluation, abstention, and drift monitoring — not as an autonomous decision-maker.*"

Part IV — Beyond Text: Multimodal Fine-Tuning

12. The Universal Recipe

Here's the unifying idea that makes audio, vision, and video feel like one subject instead of three: **almost every modern model is an encoder that turns raw input into vectors, optionally a projector that maps those vectors into a shared space, and a head or decoder that produces the output.** Fine-tuning is then "adapt some part of that stack to my data."

```
raw input ──► ENCODER ──► embeddings ──► [PROJECTOR] ──► HEAD / DECODER ──► output
 (ECG, image,   (1D-CNN,                   (align to a       (classifier, or
  video, text)   ViT, etc.)                 text LLM's space)  a language model)
```

Two patterns cover most of what you'll build:

1. **Encoder + classifier head** (discriminative). An ECG encoder → "arrhythmia / normal"; an X-ray encoder → "fracture / no fracture." You fine-tune the encoder (fully, or with PEFT) and/or a small head. This is the workhorse for detection/classification.
2. **Encoder + projector + LLM** (generative, multimodal). A vision encoder's features are projected into a language model's embedding space so the LLM can "see." This is how vision-language models (LLaVA-style, MedGemma) produce *text* about an image — reports, answers, explanations.

The transfer-learning spectrum — the same choices as language LoRA — applies to every modality:

- **Linear probing** — freeze the encoder, train only a new head. Fastest, least data, lowest ceiling. Great baseline.
- **Full fine-tuning** — unfreeze everything. Best ceiling, most data/compute, most forgetting risk.
- **PEFT (LoRA/adapters)** — the middle path, and increasingly the default in medical imaging because clinical datasets are small.

> **Interview tip.** If asked "how is fine-tuning a vision or ECG model different from an LLM?", say: "*Conceptually it isn't — it's still encoder + head, transfer learning, and the same overfitting/imbalance discipline. What changes is the encoder architecture and the input preprocessing. The strategy questions (freeze vs. LoRA vs. full, how to handle class imbalance, how to evaluate) are identical.*" That framing is exactly what they're testing.

13. Physiological Signals: ECG → Arrhythmia Detection

Detecting cardiac arrhythmia from an ECG (e.g., atrial fibrillation) is the canonical 1D biomedical-signal task. It's a superb interview topic because it exercises signal preprocessing, class imbalance, and the encoder-plus-head recipe.

13.1 The data and the task

An ECG is a time series of voltages, usually **12-lead** (12 channels) sampled at 100–500 Hz. Landmark public datasets: **PTB-XL** (~21,000 clinical 12-lead recordings with diagnostic labels), the **MIT-BIH Arrhythmia Database** (beat-level labels, the classic benchmark), and the **PhysioNet/CinC Challenge** sets. The task can be **beat-level** (classify each heartbeat) or **recording-level** (does this strip contain AF?).

13.2 Preprocessing (where signal projects live or die)

- **Bandpass filter** (e.g., 0.5–40 Hz) to remove baseline wander and high-frequency noise.
- **Resample** to a common rate so models transfer across datasets.
- **Normalize** per-lead (z-score).
- **Segment** into fixed windows; for beat classification, locate R-peaks first (the classic **Pan–Tompkins** algorithm) and window around them.
- **Handle imbalance** — dangerous arrhythmias are rare; plan for it (Chapter 9).

```
import numpy as np
from scipy.signal import butter, filtfilt

def preprocess_ecg(sig, fs=500, target_fs=250, low=0.5, high=40.0):
    # sig: (leads, samples). Bandpass then z-score per lead.
    b, a = butter(3, [low/(fs/2), high/(fs/2)], btype="band")
    sig = filtfilt(b, a, sig, axis=-1)
    # naive resample by interpolation
    n_new = int(sig.shape[-1] * target_fs / fs)
    x_old = np.linspace(0, 1, sig.shape[-1]); x_new = np.linspace(0, 1, n_new)
    sig = np.stack([np.interp(x_new, x_old, lead) for lead in sig])
    sig = (sig - sig.mean(axis=-1, keepdims=True)) / (sig.std(axis=-1, keepdims=True) +
1e-8)
    return sig.astype(np.float32)
```

13.3 A 1D-CNN classifier (the strong baseline)

For ECG, a **1D convolutional network** (often ResNet-style) is a robust, well-understood baseline; CNN-Transformer hybrids and pure transformers push accuracy further by modeling long-range rhythm. Here's a compact 1D ResNet in PyTorch:

```python
import torch, torch.nn as nn

class ResBlock1D(nn.Module):
    def __init__(self, c_in, c_out, stride=1):
        super().__init__()
        self.conv1 = nn.Conv1d(c_in, c_out, 7, stride, padding=3, bias=False)
        self.bn1   = nn.BatchNorm1d(c_out)
        self.conv2 = nn.Conv1d(c_out, c_out, 7, 1, padding=3, bias=False)
        self.bn2   = nn.BatchNorm1d(c_out)
        self.down  = (nn.Sequential(nn.Conv1d(c_in, c_out, 1, stride, bias=False),
                                    nn.BatchNorm1d(c_out))
                      if (stride != 1 or c_in != c_out) else nn.Identity())
        self.act = nn.ReLU(inplace=True)
    def forward(self, x):
        r = self.down(x)
        x = self.act(self.bn1(self.conv1(x)))
        x = self.bn2(self.conv2(x))
        return self.act(x + r)

class ECGResNet(nn.Module):
    def __init__(self, n_leads=12, n_classes=5):
        super().__init__()
        self.stem = nn.Sequential(nn.Conv1d(n_leads, 64, 15, 2, 7, bias=False),
                                  nn.BatchNorm1d(64), nn.ReLU(inplace=True))
        self.layers = nn.Sequential(
            ResBlock1D(64, 64), ResBlock1D(64, 128, stride=2),
            ResBlock1D(128, 256, stride=2), ResBlock1D(256, 256, stride=2))
        self.head = nn.Sequential(nn.AdaptiveAvgPool1d(1), nn.Flatten(),
                                  nn.Dropout(0.3), nn.Linear(256, n_classes))
    def forward(self, x):           # x: (batch, leads, samples)
        return self.head(self.layers(self.stem(x)))
```

13.4 Training with class imbalance

```python
import torch, torch.nn as nn

# Weight the loss by inverse class frequency so rare arrhythmias aren't ignored.
class_counts = torch.tensor([8000., 500., 1200., 300., 900.])   # example per-class counts
weights = (class_counts.sum() / (len(class_counts) * class_counts))
criterion = nn.CrossEntropyLoss(weight=weights)

model = ECGResNet(n_leads=12, n_classes=5)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)

def train_step(x, y):               # x:(B,12,T)  y:(B,)
    model.train(); opt.zero_grad()
    loss = criterion(model(x), y)
    loss.backward(); opt.step()
    return loss.item()
# Evaluate with AUPRC + per-class recall, NOT accuracy (see Ch. 9/11).
```

13.5 Foundation models for ECG

The field now has **ECG foundation models** (e.g., ECGFounder, pretrained on 10M+ recordings; transformer "heart-language" models that tokenize heartbeats). The play is the same as language: **fine-tune the foundation model** on your labeled set (it needs far less data than training from scratch), or **distill** a big teacher into a compact 1D-CNN student for cheap edge deployment on a wearable.

Interview tip. For "how would you build an arrhythmia detector?", walk the pipeline: data (PTB-XL/MIT-BIH) → preprocessing (filter, resample, normalize, segment) → model (1D-ResNet baseline, or fine-tune an ECG foundation model) → **imbalance-aware loss + AUPRC evaluation** → calibration + abstention for clinical use. Ending on evaluation and safety is what distinguishes a senior answer.

14. Medical Vision: X-ray → Fracture & Finding Detection

Reading an X-ray for fractures (or chest findings) is the canonical 2D medical-imaging task, and the transfer-learning story is textbook.

14.1 Data and preprocessing

Public datasets: **ChestX-ray14** (112k frontal chest X-rays, 14 findings), **CheXpert**, **MURA** (musculoskeletal radiographs, the go-to for fracture/abnormality). Preprocessing: convert **DICOM** to arrays (respect windowing), resize, normalize to the encoder's expected statistics, and augment conservatively (rotation/flip only where clinically valid — a flipped chest X-ray puts the heart on the wrong side!).

```
import torch, torchvision.transforms as T
# pydicom to read DICOM; here we assume an array `img` in [0,1], HxW.
train_tf = T.Compose([
    T.ToPILImage(), T.Resize((224, 224)),
    T.RandomRotation(7), T.RandomHorizontalFlip(p=0.0),   # flip OFF for chest X-rays
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),  # ImageNet stats
])
```

14.2 Transfer learning with a pretrained backbone

The standard recipe: take an ImageNet- (or medically-) pretrained backbone (ResNet, or a Vision Transformer), replace the classifier head, and fine-tune. **CheXNet** (a DenseNet fine-tuned on ChestX-ray14) is the classic example.

```
import torch, torch.nn as nn, torchvision

def build_xray_model(n_classes=2, freeze_backbone=False):
    m = torchvision.models.resnet50(weights="IMAGENET1K_V2")
    if freeze_backbone:                       # linear probing: train only the head
        for p in m.parameters(): p.requires_grad = False
    m.fc = nn.Linear(m.fc.in_features, n_classes)   # new head (fracture / no-fracture)
    return m

model = build_xray_model(n_classes=2, freeze_backbone=False)
opt   = torch.optim.AdamW([p for p in model.parameters() if p.requires_grad],
                          lr=3e-4, weight_decay=1e-4)
criterion = nn.CrossEntropyLoss(weight=torch.tensor([1.0, 3.0]))  # up-weight the rare
"fracture"
```

14.3 Using a medical foundation encoder + LoRA

For small clinical datasets, start from a **medical** encoder — **MedSigLIP** (the ~400M encoder behind MedGemma) or a TorchXRayVision model — and PEFT-fine-tune it. Benchmarks show LoRA/BitFit/IA3-style PEFT is competitive with full fine-tuning on

medical imaging while training a fraction of the parameters and resisting overfitting on tiny datasets — the same logic as language LoRA.

14.4 Evaluation and interpretability

Report **AUROC/AUPRC per finding**, sensitivity at a clinically chosen operating point, and **subgroup** performance. For trust and debugging, add **saliency/Grad-CAM** overlays so a radiologist can see *where* the model looked — and so you catch the infamous failure where a model "detects pneumonia" by keying on a hospital's text marker burned into the image rather than the lungs.

> **Pitfall (shortcut learning).** Medical imaging models love spurious shortcuts — scanner artifacts, laterality tokens, body-position markers. Always inspect saliency maps and evaluate on **external** data from a different site before believing your numbers.

---

15. Video: Understanding Patient Behavior

Video adds **time** to vision: you're classifying or describing *sequences* of frames — seizure detection, gait/fall analysis, pain or agitation from facial and body cues, sleep behavior. Harder to label, heavier to compute, and privacy-sensitive (faces).

15.1 The three architectural approaches

1. **Frame features + temporal model** — run a 2D CNN/ViT per frame, then aggregate over time with an LSTM/Transformer or temporal pooling. Simple, strong, cheap. Great default.
2. **3D CNNs / video transformers** — convolve or attend over space *and* time jointly (I3D, SlowFast, VideoMAE, TimeSformer). Higher ceiling, more compute.
3. **Pose-based** — first extract skeletal keypoints (e.g., with a pose estimator), then model the *pose sequence*. Excellent for behavior/gait, and it **discards appearance**, which is a privacy win in clinical video.

```python
import torch, torch.nn as nn, torchvision

class VideoBehaviorNet(nn.Module):
    """Per-frame CNN features -> temporal Transformer -> behavior class."""
    def __init__(self, n_classes=4, d=512, n_heads=8, n_layers=2):
        super().__init__()
        backbone = torchvision.models.resnet18(weights="IMAGENET1K_V1")
        self.feat = nn.Sequential(*list(backbone.children())[:-1])  # (B,512,1,1) per frame
        enc = nn.TransformerEncoderLayer(d, n_heads, batch_first=True)
        self.temporal = nn.TransformerEncoder(enc, n_layers)
        self.cls = nn.Sequential(nn.LayerNorm(d), nn.Linear(d, n_classes))
    def forward(self, x):                # x: (B, T, C, H, W)
        B, T = x.shape[:2]
        f = self.feat(x.flatten(0, 1)).flatten(1)      # (B*T, 512)
        f = f.view(B, T, -1)                           # (B, T, 512)
        h = self.temporal(f).mean(dim=1)               # temporal pooling -> (B, 512)
        return self.cls(h)
```

15.2 Practical realities

- **Labeling** is the bottleneck — temporal annotation (when does the event start/stop?) is expensive; weak/self-supervised pretraining on unlabeled clinical video helps.
- **Compute** — sample frames (you rarely need 30 fps); clip into short windows.
- **Privacy** — faces are PHI-adjacent. Pose-only pipelines, on-device processing, and on-prem deployment matter even more here than for text.
- **Fine-tuning** — start from a video-pretrained backbone (VideoMAE/SlowFast) and fine-tune the head or the whole net; the transfer-learning spectrum is unchanged.

> **Interview tip.** If handed "detect patient falls from ward video," propose the **pose-based** route first: pose estimation → temporal model on keypoints. It's accurate, cheaper, and *privacy-preserving* (no faces stored) — showing you weigh ethics and cost, not just accuracy.

---

16. Vision-Language Models: Fine-Tuning a Medical VLM

The generative frontier of healthcare AI is the **vision-language model (VLM)**: give it an image and a question, get *text* back — a radiology report, a visual Q&A answer, an explanation. This is Chapter 12's "encoder + projector + LLM" pattern realized.

16.1 How a VLM is wired

A vision encoder (e.g., a SigLIP/MedSigLIP ViT) turns the image into patch embeddings; a small **projector** (an MLP) maps them into the language model's token-embedding space; those visual "tokens" are prepended to the text tokens, and the LLM generates text conditioned on both. **MedGemma** is exactly this, built on Gemma 3 with MedSigLIP as its eyes.

16.2 Fine-tuning MedGemma with LoRA

You fine-tune a VLM on *(image, prompt, target-text)* triples — for example, an X-ray plus "Describe the findings" plus the ground-truth report. Mechanically it's SFT with LoRA, where the image is passed through the processor alongside the text.

```
import torch
from transformers import AutoProcessor, AutoModelForImageTextToText
from peft import LoraConfig, get_peft_model

MODEL = "google/medgemma-4b-it"          # medical VLM (Gemma 3 + MedSigLIP)
proc  = AutoProcessor.from_pretrained(MODEL)
model = AutoModelForImageTextToText.from_pretrained(
    MODEL, dtype=torch.bfloat16, device_map="auto")

# LoRA on the language-model projections (and optionally the projector); keep the
# heavy vision encoder frozen to save memory and avoid overfitting small data.
model = get_peft_model(model, LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05, task_type="CAUSAL_LM",
    target_modules=["q_proj","k_proj","v_proj","o_proj"]))
model.print_trainable_parameters()

def collate(example):   # example: {"image": PIL, "prompt": str, "answer": str}
    messages = [{"role":"user","content":[
                    {"type":"image"},
                    {"type":"text","text": example["prompt"]}]},
                {"role":"assistant","content":[
                    {"type":"text","text": example["answer"]}]}]
    batch = proc.apply_chat_template(messages, tokenize=True,
                                     add_generation_prompt=False,
                                     return_dict=True, return_tensors="pt",
                                     images=[example["image"]])
    batch["labels"] = batch["input_ids"].clone()     # mask prompt tokens in practice
    return batch
# Feed `collate` outputs to a Trainer / training loop exactly like text SFT.
```

16.3 Evaluating a medical VLM

For report generation, fluency metrics (BLEU/ROUGE) are weak; use **clinical-entity metrics like RadGraph F1** (are the right findings and relations present?) plus human/expert review and **hallucination checks** (did it invent a finding not in the image?). After fine-tuning, MedGemma reportedly reaches state-of-the-art chest-X-ray report generation — a concrete data point worth citing.

> **Interview tip.** The senior framing of multimodal healthcare AI: "*Discriminative models (ECG/X-ray classifiers) give you a calibrated number for triage; generative VLMs give you a draft report a clinician edits. Different risk profiles — the classifier needs AUPRC and calibration; the VLM needs faithfulness and hallucination control. I'd use each where its failure mode is acceptable.*"

Part V — Reasoning ("Thinking") Models

17. What Makes a Model Reason

"Thinking models" — OpenAI's o-series, DeepSeek-R1, and their kin — spend extra tokens *deliberating* before answering, typically inside a visible or hidden `<think>...</think>` scratchpad, then emit a final answer. They dramatically improve math, code, and multi-step logic. Understanding how they're made is now a core interview topic.

17.1 Three ways to get reasoning into a model

1. **Chain-of-thought prompting** (no training) — just ask the model to "think step by step." Free, but limited by the base model's latent ability.
2. **Distillation (SFT on reasoning traces)** — take a strong reasoner (e.g., R1), generate long chain-of-thought solutions, and **fine-tune a smaller model on those traces**. Simple, stable, and remarkably effective: DeepSeek showed that *distilling* R1 into Qwen/Llama bases often beats running RL directly on those small models. This is the pragmatic path for most teams and most hardware.
3. **Reinforcement learning with verifiable rewards (RLVR)** — the technique that *created* the reasoning behavior in the first place, and the deepest of the three.

17.2 RLVR and GRPO — the engine behind R1

The breakthrough insight: for tasks where you can **automatically check correctness** (math has a known answer; code either passes tests or doesn't), you don't need a learned reward model or human labels. You let the model attempt a problem, **verify** the answer with a deterministic checker, and reward it for being right. This is **Reinforcement Learning with Verifiable Rewards (RLVR)**.

The algorithm of choice is **Group Relative Policy Optimization (GRPO)**. Its trick, relative to PPO: instead of training a separate value/critic network (as big as the model), it samples a **group** of answers per prompt and computes each answer's advantage *relative to the group's average reward*. Cheaper, simpler, and stable. DeepSeek-R1-Zero showed that GRPO with purely verifiable rewards — even with **no SFT warm-up** — makes reasoning *emerge*, including spontaneous self-reflection ("wait, let me reconsider…", the famous "aha moment").

The reward is usually two simple rule-based parts:

- **Accuracy reward** — did the final answer (in a required format, e.g., in `\boxed{}`) match the known solution?
- **Format reward** — did the model put its reasoning in `<think>...</think>` and its answer where expected?

```python
# A verifiable reward function — the heart of RLVR. Deterministic, no reward model.
import re

def reward_fn(prompt, completion, ground_truth):
    reward = 0.0
    # (1) format: reasoning in <think>...</think> then a boxed answer
    if re.search(r"<think>.*?</think>", completion, re.DOTALL):
        reward += 0.1
    # (2) correctness: extract \boxed{...} and compare to the known answer
    m = re.search(r"\\boxed\{([^}]*)\}", completion)
    if m and m.group(1).strip() == str(ground_truth).strip():
        reward += 1.0
    return reward
```

This is precisely why domains with **checkable correctness** are the sweet spot for reasoning RL — math, code, formal logic. It's also why a rule-governed language like **Sanskrit** (Panini's grammar makes sandhi, segmentation, and meter programmatically verifiable) is an unusually good candidate for verifiable-reward training, where English is not.

> **Interview tip.** Distinguish crisply: **RLHF** learns a reward *model* from human preferences (good for fuzzy "helpfulness"); **RLVR** uses a deterministic *verifier* for binary correctness (good for math/code/logic). GRPO is the value-function-free RL algorithm both R1-Zero and R1 used. And note the honest nuance from recent research: RLVR seems to mostly *amplify and reconfigure* reasoning already latent in the base model rather than inject brand-new knowledge — so base-model quality still matters enormously.

---

18. Building a Thinking Model, Step by Step

Two realistic paths given real hardware. Pick by budget.

18.1 Path A — Distillation (recommended default)

1. **Collect reasoning traces.** Take a strong open reasoner, prompt it on thousands of problems in your domain, and keep the full `<think>` + answer traces (filter to *correct* ones using a verifier).
2. **SFT on the traces** using the Chapter 5 pipeline — the model learns to imitate the *shape* of good reasoning. That's it. No RL loop, stable, single-GPU-friendly.

```
{"prompt": "If a dose is 5 mg/kg for a 12 kg child, total dose?",
 "completion": "<think>5 mg/kg × 12 kg = 60 mg. Units: mg/kg × kg = mg.</think>\n\\boxed{60 mg}"}
```

18.2 Path B — RLVR with GRPO (when you want emergent reasoning)

TRL provides `GRPOTrainer`. You supply a reward function (your verifier) and a dataset of prompts with checkable answers.

```
from datasets import load_dataset
from trl import GRPOTrainer, GRPOConfig

def reward_fn(completions, ground_truth, **kw):
    import re
    out = []
    for c, gt in zip(completions, ground_truth):
        r = 0.1 if re.search(r"<think>.*?</think>", c, re.DOTALL) else 0.0
        m = re.search(r"\\boxed\{([^}]*)\}", c)
        r += 1.0 if (m and m.group(1).strip() == str(gt).strip()) else 0.0
        out.append(r)
    return out

ds = load_dataset("json", data_files="math_prompts.jsonl", split="train")  # prompt, ground_truth
trainer = GRPOTrainer(
    model="Qwen/Qwen2.5-3B-Instruct",
    reward_funcs=reward_fn,
    args=GRPOConfig(output_dir="./r1-style", num_generations=8,   # group size for GRPO
                    per_device_train_batch_size=1, gradient_accumulation_steps=8,
                    learning_rate=1e-6, bf16=True, report_to="none"),
    train_dataset=ds)
trainer.train()
```

18.3 Watch-outs

- **Reward hacking** — models exploit any gap in your verifier. If it catches 60% of errors, the model finds the other 40%. Make verifiers tight and comprehensive.

- **Exploration collapse** — if every sampled answer in a group is wrong, the group advantage is zero and there's no gradient. Curriculum (easy → hard) and starting from a decent SFT'd model help.
- **Cost of long outputs** — reasoning models emit many tokens, which is expensive at *inference* too. This directly motivates the deployment choices in Part VI (NVIDIA Dynamo is explicitly tuned for the long-output reasoning workload).

> **Interview tip.** For most companies and most hardware, **distillation beats DIY RL**: cheaper, stable, and often better for small models. Recommend RLVR/GRPO when you need capability beyond what any teacher offers, you have a solid verifier, and you can afford the RL compute. Signaling that judgment — not just naming GRPO — is what lands.

Part VI — Private Deployment on NVIDIA Hardware

19. Inference Optimization

Training is a one-time cost; **inference is forever.** In healthcare, inference must also stay **private and on-prem**. This part covers making models fast and cheap, then the NVIDIA stack that serves them.

19.1 Why generation is slow, and the two phases

LLM generation has two distinct phases with different bottlenecks:

- **Prefill** — process the whole prompt in parallel. **Compute-bound** (great GPU utilization).
- **Decode** — generate tokens one at a time, each depending on the last. **Memory-bandwidth-bound** — you re-read the whole model and the growing **KV cache** for every single token. This is why token generation feels slow and why memory bandwidth (not FLOPs) often decides throughput.

Understanding this split explains almost every optimization below — including **disaggregated serving**, which runs prefill and decode on *different* GPUs tuned for each.

19.2 Quantization for inference

Shrink weights (and sometimes activations/KV cache) to fewer bits to cut memory and bandwidth:

- **INT8 / FP8** — ~2× smaller than fp16 with minimal quality loss; FP8 is well-supported on recent NVIDIA GPUs.
- **INT4 (GPTQ, AWQ)** — ~4× smaller; small quality hit; excellent for fitting big models on modest GPUs.
- **FP4 / NVFP4** — Blackwell-generation 4-bit float; NVIDIA reports sub-1% accuracy loss with big throughput gains.

Post-training quantization for serving (distinct from QLoRA, which was for *training*) is typically done with **NVIDIA TensorRT Model Optimizer** or AWQ/GPTQ toolkits, producing a checkpoint your server loads.

19.3 The other big levers

- **KV-cache management** — **PagedAttention** (from vLLM) stops KV-cache memory fragmentation and enables much larger batch sizes; the single biggest throughput win for serving.
- **Continuous (in-flight) batching** — dynamically add/remove requests from a running batch instead of waiting for a fixed batch to finish. Massive utilization gains under real traffic.

- **Speculative decoding** — a small "draft" model proposes several tokens, the big model verifies them in one pass; 2–3× faster decode with identical output distribution.
- **FlashAttention** — an IO-aware attention kernel that's faster and more memory-efficient; on by default in modern stacks.

> **Interview tip.** If asked "how do you make LLM inference faster/cheaper on-prem?", organize your answer as: **(1) quantize** (FP8/FP4/INT4), **(2) batch** (continuous batching + PagedAttention), **(3) speculate** (draft model), **(4) disaggregate** prefill/decode for reasoning workloads — and pick a server that does these for you (next chapter).

---

20. The NVIDIA Serving Stack

For private enterprise inference on your own NVIDIA hardware, there's a layered ecosystem. Know what each layer is *for* — a very common interview probe.

20.1 The layers, bottom to top

- **CUDA / cuDNN** — the compute foundation. You rarely touch it directly.
- **TensorRT** — a graph compiler/optimizer that turns a trained network into a highly optimized inference **engine** for a specific GPU (fused kernels, quantization, tuned memory). The general-purpose engine builder — great for the CNN/ECG/X-ray models of Part IV.
- **TensorRT-LLM** — TensorRT specialized for LLMs (paged KV cache, in-flight batching, FP8/FP4, LoRA serving, speculative decoding). PyTorch-native API. This is the LLM optimization engine.
- **Triton Inference Server** (now **Dynamo-Triton**) — the **multi-framework model server**: serves TensorRT, PyTorch, ONNX, Python, and more behind one endpoint, with dynamic batching, concurrent execution, model **ensembles** (chain preprocessing + model + postprocessing), and metrics. This is where you deploy your *non-LLM* models (ECG classifier, X-ray CNN) and mixed pipelines.
- **NVIDIA Dynamo** — the newest layer: a **distributed, datacenter-scale** LLM serving framework for models too big for one GPU and for high-volume/reasoning workloads. It adds **disaggregated serving** (prefill and decode on different GPUs), **KV-cache-aware routing**, and KV offload across GPU/CPU/SSD. It's backend-agnostic (works with TensorRT-LLM, vLLM, SGLang). Explicitly optimized for **reasoning models**, which emit many tokens.
- **NVIDIA NIM (NVIDIA Inference Microservices)** — the easy button: a **Docker container exposing an OpenAI-compatible API** for a model, with TensorRT-LLM + Triton pre-optimized inside. `pull → run → serve` in hours instead of a multi-week MLOps project. Part of **NVIDIA AI Enterprise** (per-GPU license, long-term support).

Open-source alternatives you should also name: **vLLM** and **SGLang** (superb, widely used LLM servers) — Dynamo interoperates with both, so it's not all-or-nothing NVIDIA.

20.2 How to choose

- **Just serve a standard LLM privately, fast? → NIM** (or vLLM if you want fully open-source).
- **Serve custom/non-LLM models (ECG, X-ray) or multi-step pipelines? → Triton** with a TensorRT engine.
- **Serve huge or reasoning models across many GPUs at scale? → Dynamo** (with TensorRT-LLM or vLLM under it).
- **Squeeze maximum single-model performance?** → compile with **TensorRT-LLM** and quantize with **TensorRT Model Optimizer**.

20.3 Serving an LLM with a NIM (sketch)

```
# NIM exposes an OpenAI-compatible endpoint; your app code barely changes.
docker run --gpus all -p 8000:8000 \
  -e NGC_API_KEY=$NGC_API_KEY \
  nvcr.io/nim/meta/llama-3.1-8b-instruct:latest

# Then call it like OpenAI:
curl http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"meta/llama-3.1-8b-instruct",
       "messages":[{"role":"user","content":"Summarize this discharge note..."}]}'
```

20.4 Serving your fine-tuned LoRA adapter

TensorRT-LLM and NIM support **multi-LoRA serving** — keep one base model in memory and hot-swap many small adapters per request. For a hospital with several fine-tuned specialties (radiology, cardiology, discharge-summary), this serves them all from one base at near-zero extra memory. Alternatively, `merge_and_unload()` the adapter into the base for a single standalone checkpoint and serve that.

> **Interview tip.** A clean one-liner: "*NIM is the pre-packaged OpenAI-compatible microservice; underneath it's TensorRT-LLM for optimization and Triton for serving; Dynamo is the distributed layer above for scale and reasoning-model workloads. For my custom ECG/X-ray models I'd use Triton directly with a TensorRT engine.*"

---

21. Serving Non-LLM & Multimodal Models with Triton

Your ECG and X-ray models aren't LLMs, so they take the **Triton** path. The workflow:

1. **Export** the trained PyTorch model to **ONNX** (or build a **TensorRT** engine from it).
2. Lay out a **model repository** (a directory Triton reads) with a `config.pbtxt` describing inputs/outputs, batching, and instance count.
3. **Run Triton**, which serves HTTP/gRPC with dynamic batching and metrics.

```
model_repository/
└── ecg_arrhythmia/
    ├── config.pbtxt
    └── 1/
        └── model.onnx
```

```
# config.pbtxt — dynamic batching turns many small clinical requests into efficient batches.
name: "ecg_arrhythmia"
platform: "onnxruntime_onnx"
max_batch_size: 64
input  [ { name: "ecg"    data_type: TYPE_FP32  dims: [ 12, 2500 ] } ]  # 12 leads × samples
output [ { name: "logits" data_type: TYPE_FP32  dims: [ 5 ] } ]
dynamic_batching { max_queue_delay_microseconds: 5000 }
instance_group [ { count: 2  kind: KIND_GPU } ]
```

Ensembles let you push DICOM/signal preprocessing and postprocessing into Triton too, so the client sends raw data and gets a clean prediction — one endpoint, GPU-accelerated end to end. For a **multimodal VLM** (MedGemma), you'd serve it via TensorRT-LLM/Triton (it's a Gemma-family LLM with a vision tower) or a NIM if available.

---

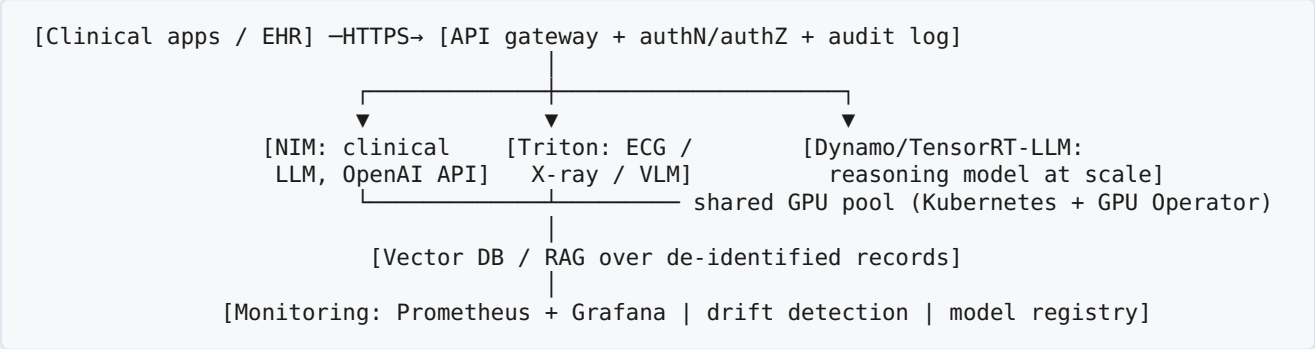
22. On-Prem Architecture, Security & MLOps

Pulling it together into a system a hospital would actually run.

22.1 Why on-prem at all

For healthcare the driver is usually **data sovereignty**: PHI can't leave the institution or go to a public API, and the DGX-class box's **unified memory keeps proprietary training data on-premises**. On-prem also gives predictable latency and cost, and avoids per-token cloud bills for high-volume inference.

22.2 A reference architecture



Key pieces: **Kubernetes** with the **NVIDIA GPU Operator** for scheduling; **Prometheus/Grafana** for metrics (Triton and Dynamo export them natively); a **model registry** for versioning; and a **RAG layer** over de-identified records so the LLM is grounded in real patient context without memorizing PHI in weights.

22.3 Security and compliance in production

- **Isolation** — private network, no egress for PHI; encrypt at rest and in transit.
- **Access control & audit** — every inference on patient data logged (who, what, when) for HIPAA.
- **PII/PHI at inference** — scrub logs and prompts; don't accidentally persist PHI in traces.
- **Model governance** — version models *and* datasets; for regulated use, a **Predetermined Change Control Plan** so you can update models within an approved envelope.
- **Guardrails** — input/output filtering, clinical safety checks, and human-in-the-loop for high-risk outputs.

22.4 MLOps: the model isn't done when it trains

- **Monitor drift** — inputs and performance shift as populations, scanners, and coding practices change; alert and schedule re-training.

- **Canary / shadow deploys** — run a new model in shadow against live traffic before promoting it.
- **Feedback loop** — capture clinician corrections as new training data (with governance) to improve over time.

> **Interview tip.** Tie the whole course together: "*Fine-tuning is one step in a lifecycle — data governance → PEFT training → rigorous, calibrated, subgroup-aware evaluation → optimized private serving on NVIDIA (NIM/Triton/Dynamo) → monitoring and governed re-training. In healthcare, privacy and evaluation dominate; the training loop is the easy part.*"

Appendix A — DGX Spark / NVIDIA Environment Setup

The GB10 (Grace Blackwell, ARM64, CUDA 13, 128 GB unified memory) is an excellent personal fine-tuning box, with two gotchas:

1. **Use NVIDIA's PyTorch container, not generic pip wheels** — the container has the Blackwell-optimized PyTorch, FlashAttention, and Triton. `bash docker pull nvcr.io/nvidia/pytorch:25.11-py3 docker run --gpus all -it --shm-size=16g \ -v $HOME/work:/work -v $HOME/.cache/huggingface:/root/.cache/huggingface \ nvcr.io/nvidia/pytorch:25.11-py3 bash pip install transformers peft trl datasets accelerate bitsandbytes`
2. **Use bf16, not fp16** — the Blackwell + fp16 combination has documented numerical issues; bf16 is the recommended precision for both training and 4-bit compute dtype.

Memory math: with 128 GB unified memory you can QLoRA-fine-tune up to ~70B, full-fine-tune small models, and run inference up to ~200B. Token generation is memory-bandwidth-bound, so it's a superb *development* box; final high-throughput serving belongs on datacenter GPUs.

Appendix B — Interview Question Bank

Short, sharp answers to rehearse.

Q: Fine-tuning vs. RAG vs. prompting — when each? Prompting first (free). RAG when the gap is *knowledge* (facts, private docs). Fine-tuning when the gap is *behavior/format/style/skill*, or to run a smaller model. They compose.

Q: Explain LoRA and why it works. Freeze W; train two low-rank matrices whose product is added to W. Works because the adaptation needed to specialize a model is low-rank; captures most of the benefit at <1% of parameters, with mild forgetting since the base is frozen.

Q: QLoRA? LoRA on a 4-bit (nf4) quantized frozen base; adapters stay bf16. ~4× base memory saving, small speed cost from de-quantizing during compute. Lets you fine-tune big models on one GPU.

Q: SFT loss? Token-level cross-entropy (next-token prediction), with prompt tokens masked so loss is computed on the completion only.

Q: DPO vs. RLHF vs. RLVR? RLHF learns a reward model from human preferences + PPO. DPO skips the reward model with a direct preference classification loss (stable, cheap). RLVR uses a deterministic verifier (binary correct/incorrect) — no reward model — ideal for math/code; GRPO is the value-free RL algorithm (used by DeepSeek-R1).

Q: How do reasoning models get built? CoT prompting (free), distillation (SFT on a strong reasoner's traces — best for small models), or RLVR/GRPO with verifiable rewards (emergent reasoning, the "aha moment"). Distillation is the pragmatic default.

Q: Class imbalance in medical data? Don't use accuracy. Use AUPRC/recall/F1; class-weighted or focal loss; careful resampling; choose an operating point by clinical cost; evaluate per-subgroup and on external data.

Q: How do you evaluate a fine-tuned model? Held-out, decontaminated test set; task-appropriate metrics (chrF/COMET for Indic MT, RadGraph F1 for reports, AUPRC for imbalanced classification); LLM-as-judge with bias controls for open-ended text; report confidence intervals; evaluate the deployed system (weights + template + RAG).

Q: Multilingual fine-tuning — what's hard for Tamil/Urdu/Sanskrit? Tokenizer fertility and script specifics (Devanagari abugida syllables; Urdu Perso-Arabic RTL + diacritics). Fix: Indic/multilingual base, or tokenizer extension + continued pretraining. SFT alone won't teach a language the base never saw.

Q: NIM vs. Triton vs. Dynamo vs. TensorRT-LLM? TensorRT-LLM optimizes an LLM; Triton serves any-framework models (your ECG/X-ray nets) with batching/ensembles; Dynamo is distributed serving for huge/reasoning models (disaggregated prefill/decode, KV routing); NIM is the pre-packaged OpenAI-compatible container using TensorRT-LLM+Triton inside.

Q: Why on-prem for healthcare? PHI data sovereignty (can't leave the institution / go to public APIs), predictable latency/cost, audit and compliance control. Keep data in the building; serve on your own NVIDIA hardware.

Q: Catastrophic forgetting — what and how to mitigate? Fine-tuning erodes prior skills. Mitigate with PEFT (frozen base), lower LR / fewer epochs, mixing in general data, and evaluating retained capabilities, not just the new task.

Appendix C — Glossary

Abugida — a script where consonants carry an inherent vowel modified by signs (Devanagari). **AUPRC** — area under precision-recall curve; preferred under class imbalance. **Chat template** — the tokenizer's exact turn-marker format for instruct models. **Continuous batching** — dynamically adding/removing requests from a running inference batch. **Decode/Prefill** — the two LLM inference phases (bandwidth-bound vs. compute-bound). **Disaggregated serving** — running prefill and decode on separate GPUs. **Distillation** — training a student on a teacher's outputs. **Fertility** — average tokens per word (tokenizer efficiency). **GRPO** — Group Relative Policy Optimization; value-free RL for reasoning. **KV cache** — stored keys/values so past tokens aren't recomputed each decode step. **LoRA/QLoRA** — low-rank adapters / on a 4-bit base. **NIM** — NVIDIA Inference Microservice (OpenAI-compatible container). **PagedAttention** — paged KV-cache memory management (vLLM). **PEFT** — parameter-efficient fine-tuning. **PHI** — protected health information. **RLVR** — RL with verifiable rewards. **SFT** — supervised fine-tuning. **Speculative decoding** — draft-then-verify token generation. **Triton/Dynamo** — NVIDIA model servers.

Appendix D — Datasets & Models Quick Reference

Language / Indic: Sarvam-1 (Indic base, efficient tokenizer), Gemma 3 / MedGemma substrate, Qwen (tooling), Llama 3.x; benchmarks IndicGLUE, IndicXTREME, Flores-200; AI4Bharat ecosystem. **ECG:** PTB-XL (~21k 12-lead), MIT-BIH Arrhythmia DB, PhysioNet/ CinC challenges; ECG foundation models (ECGFounder). **Medical imaging:** ChestX-ray14, CheXpert, MURA; encoders MedSigLIP, TorchXRayVision, MedImageInsight; VLM MedGemma; metric RadGraph F1; toolkit MONAI. **Serving:** NVIDIA NIM, TensorRT-LLM, Triton/Dynamo-Triton, Dynamo, TensorRT Model Optimizer; open-source vLLM, SGLang.

End of course. Fly safe — and remember the refrain: the model is the easy part; the data, the evaluation, and the deployment are where the real work (and the interview points) live.